

Experiment 30: Forward Chaining Using Prolog

Aim

To implement **Forward Chaining using Prolog** and demonstrate how new facts can be derived from existing facts by applying rules sequentially.

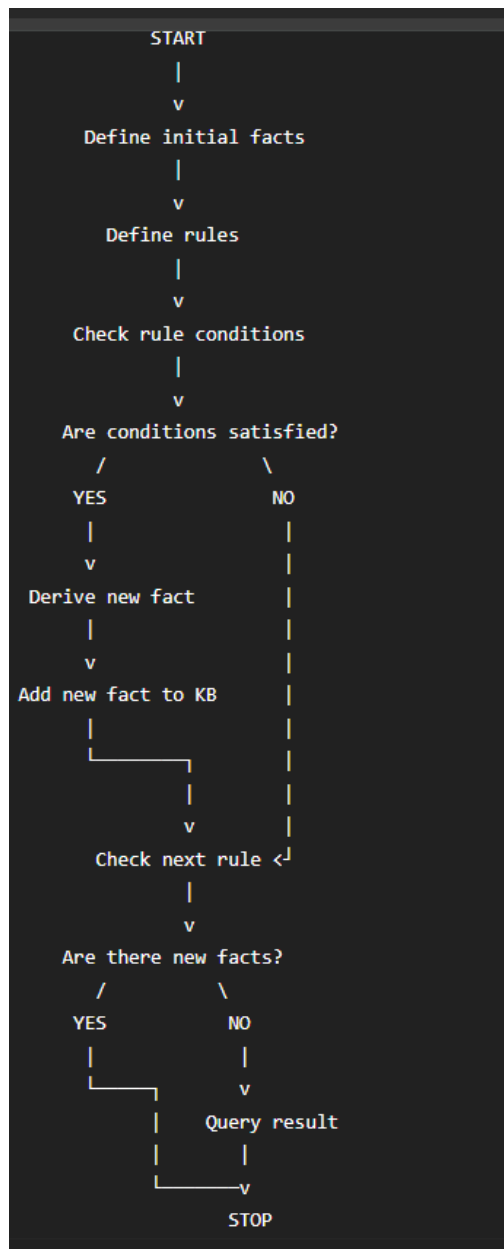
Objective

1. To understand the concept of forward chaining.
2. To represent knowledge using facts and rules.
3. To derive new information from existing facts.
4. To implement forward-style reasoning in Prolog.
5. To execute queries and verify the derived conclusions.

Algorithm

1. Start the program.
2. Define the initial facts.
3. Define rules that can derive new facts from existing facts.
4. Start with the known facts.
5. Check whether the conditions of a rule are satisfied.
6. If the conditions are satisfied, derive the new fact.
7. Continue applying rules to the newly derived facts.
8. Repeat until no new facts can be derived.
9. Query the knowledge base to verify the conclusion.
10. Display the result.
11. Stop.

Flowchart



Prolog Code

% Forward Chaining Example

% Initial facts

bird(sparrow).

has_wings(sparrow).

```
can_fly(sparrow).
```

```
% Rules
```

```
derive_animal(X) :-  
    bird(X).
```

```
derive_flying_animal(X) :-  
    bird(X),  
    has_wings(X),  
    can_fly(X).
```

```
% Final conclusion
```

```
flying_animal(X) :-  
    derive_flying_animal(X).
```

Sample Output

Query 1: Check whether Sparrow is a bird

```
?- bird(sparrow).
```

Query 2: Derive whether Sparrow is an animal

```
?- derive_animal(sparrow).
```

Query 3: Determine whether Sparrow is a flying animal

```
?- flying_animal(sparrow).
```

Query 4: Find all flying animals

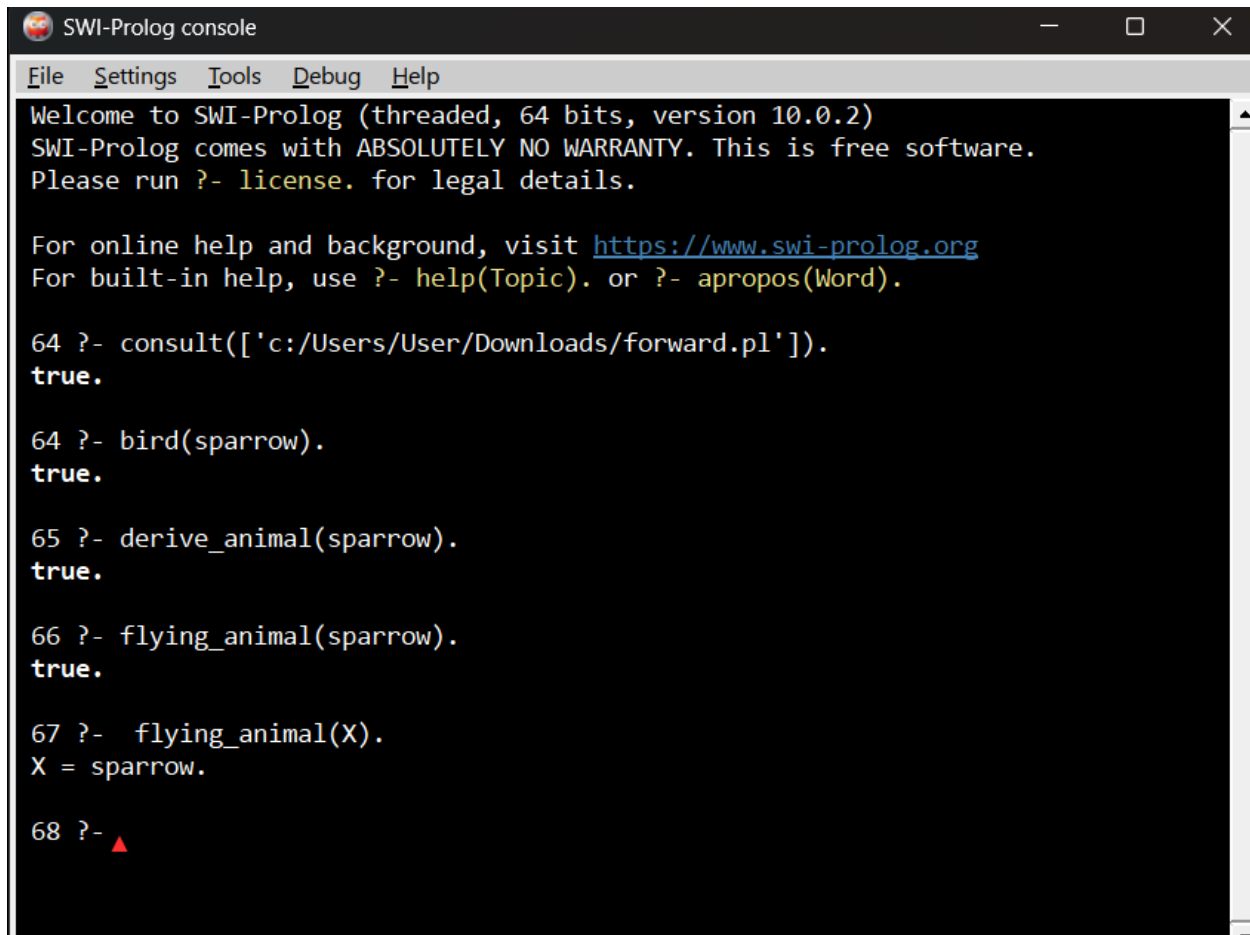
```
?- flying_animal(X).
```

Output:

```
X = sparrow.
```

Query 5: Check whether Dog is a flying animal

```
?- flying_animal(dog).
```

A screenshot of the SWI-Prolog console window. The window has a title bar with the text "SWI-Prolog console" and standard window controls. Below the title bar is a menu bar with "File", "Settings", "Tools", "Debug", and "Help". The main area is a black terminal with white text. It displays a welcome message, a license notice, and several Prolog queries and their results. The queries are: 1. consult(['c:/Users/User/Downloads/forward.pl']). 2. bird(sparrow). 3. derive_animal(sparrow). 4. flying_animal(sparrow). 5. flying_animal(X). The results are: 1. true. 2. true. 3. true. 4. true. 5. X = sparrow. The prompt "68 ?-" is followed by a red triangle cursor.

```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

64 ?- consult(['c:/Users/User/Downloads/forward.pl']).
true.

64 ?- bird(sparrow).
true.

65 ?- derive_animal(sparrow).
true.

66 ?- flying_animal(sparrow).
true.

67 ?- flying_animal(X).
X = sparrow.

68 ?- ▲
```

Result

The **Forward Chaining concept** was **successfully demonstrated using Prolog**. The program used known facts and rules to derive new conclusions and successfully determined whether a given animal is a flying animal.

Conclusion

Thus, the experiment demonstrates **forward-style reasoning**, where known facts are used to derive new information by applying rules. The resulting conclusions can then be verified through Prolog queries.

Note: Standard Prolog primarily uses backward chaining internally. Therefore, this program demonstrates the **forward reasoning concept** using Prolog rules rather than implementing a complete forward-chaining inference engine.